



C.P.R. Liceo “La Paz”

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma

Autor: Antón Faraldo Mosquera

Tutor: Jesús Ángel Pérez-Roca

• Resumen

El presente proyecto consiste en el diseño y desarrollo de “MineManager Pro”, una aplicación de escritorio inspirada en el clásico juego de lógica Buscaminas, pero evolucionada hacia un entorno moderno de gestión y auditoría. El sistema integra una interfaz gráfica interactiva y enriquecida mediante JavaFX para la experiencia del usuario, acoplada a una sólida arquitectura de persistencia de datos relacional gobernada por Hibernate. La aplicación no solo ofrece una experiencia de juego pulida (con modalidades clásica y contrarreloj soportada por dinámicas multimedia), sino que incorpora un robusto módulo de control de accesos, roles diferenciados (Jugador y Administrador Maestro), un sistema automatizado de logros en tiempo real y una infraestructura híbrida de almacenamiento local e histórico mediante archivos CSV y configuraciones JSON. El resultado es una herramienta de software que unifica el entretenimiento lógico con las prácticas de gestión, seguridad y persistencia avanzadas exigidas en el desarrollo corporativo actual.

• Abstract

This project entails the design and development of “MineManager Pro”, a desktop application inspired by the classic Minesweeper logic game, evolved into a modern management and auditing environment. The system integrates an interactive and enriched graphical user interface using JavaFX for the user experience, coupled with a solid relational data persistence architecture governed by Hibernate. The application not only offers a polished gameplay experience (featuring classic and time-attack modes supported by dynamic multimedia audio effects) but also incorporates a robust access control module, differentiated roles (Player and Master Administrator), an automated real-time achievement system, and a hybrid data infrastructure utilizing CSV files for local backups and JSON files for configuration management. The outcome is a software solution that unifies logical entertainment with the advanced management, security, and persistence practices demanded in today’s corporate software development.

• Palabras Clave

Aplicación de escritorio (Desktop Application): Software diseñado para ser instalado y ejecutado localmente en computadoras personales (sistemas operativos de sobremesa como Windows o Linux), interactuando de forma directa con los recursos físicos de la máquina cliente sin depender de un navegador web.

JavaFX: Framework gráfico utilizado para el desarrollo de la interfaz de usuario (UI), permitiendo una separación limpia de la lógica y el diseño mediante archivos FXML y controladores.

Hibernate (ORM): Framework de mapeo objeto-relacional empleado para interactuar con la base de datos de manera orientada a objetos, simplificando la persistencia de usuarios, partidas y logros sin necesidad de escribir consultas SQL nativas.

MySQL: Sistema de gestión de bases de datos relacionales utilizado para el almacenamiento persistente y estructurado de la información del sistema (historial de juego, registros de auditoría y credenciales).

Arquitectura MVC (Modelo-Vista-Controlador): Patrón de diseño de software que estructura el código del proyecto separando los datos (Model), la interfaz gráfica (Vista) y la lógica de negocio que los conecta (Controlador).

Persistencia Híbrida: Estrategia de almacenamiento que combina una base de datos relacional centralizada con copias de seguridad en formato plano (CSV) e intercambio de configuraciones parametrizadas (JSON).

Auditoría y Control de Accesos (RBAC): Mecanismo de seguridad basado en roles (Jugador y Administrador Maestro) que restringe o permite el acceso a vistas y operaciones críticas del sistema, registrando las acciones de los usuarios para su posterior supervisión en paneles administrativos.

Multimedia Interactiva y Polifonía: Capacidad del sistema para cargar y reproducir efectos de audio simultáneos independientes (clics, banderas, alertas de tensión aceleradas mediante pitch shifting) a través del motor nativo del sistema operativo, proveyendo al usuario de una respuesta o feedback sensorial inmediato controlable mediante interfaces de accesibilidad (Mute).

Mapeo de Datos Plano (CSV): Técnica de serialización de datos utilizada a través de un componente gestor (CSVManager) para exportar e importar registros históricos del juego en texto plano estructurado por delimitadores, asegurando la portabilidad de los datos fuera del entorno de red de la base de datos.

Parsing Estricto de Configuraciones (JSON): Proceso de lectura e interpretación de datos estructurados en formato ligero JavaScript Object Notation, empleado para cargar diccionarios de datos externos (filtros de palabras prohibidas, configuraciones de inicialización) de manera dinámica durante el arranque de la aplicación.

Patrón de Diseño Singleton (Instancia Única): Patrón creacional de software implementado en la infraestructura del núcleo de la aplicación (AppShell) para garantizar que una clase posea una única instancia global accesible en todo el proyecto,

centralizando el control de la navegación entre pantallas y el estado de la sesión activa del usuario.

Java Mail API (Servicio de Mensajería): Protocolo de servicios asíncronos integrado en la lógica de negocio (EmailService) que permite a la aplicación conectarse de forma segura a servidores SMTP externos para la expedición automatizada de notificaciones por correo electrónico (como la recuperación de credenciales), comunicando el sistema local con redes externas.

API Criteria de Hibernate (Consultas Dinámicas): Interfaz avanzada de persistencia utilizada en la capa de acceso a datos para construir consultas programáticas y fuertemente tipadas a la base de datos MySQL, empleando restricciones dinámicas (Restrictions, Projections) para calcular clasificaciones globales (rankings) y estadísticas en tiempo real de forma segura contra inyecciones.

Cifrado Criptográfico Hashing (Seguridad de Credenciales): Técnica de seguridad aplicada en el proceso de autenticación (LoginController, Usuario) mediante la cual las contraseñas de los usuarios no se almacenan en texto plano en la base de datos, sino transformadas mediante algoritmos de una sola vía, garantizando la confidencialidad y la protección de datos conforme a estándares profesionales.

Transiciones de Renderizado Gráfico (FX Transitions): Componentes de animación visual (TranslateTransition, ScaleTransition, FadeTransition) embebidos en los controladores de la interfaz, para aplicar interpolaciones físicas dinámicas sobre los nodos de la vista (como el efecto confeti, la sacudida por error del indicador de minas o el despliegue elástico del panel de fin de partida).

Inyección de Dependencias FXML (@FXML): Mecanismo de desacoplamiento de software provisto por JavaFX que vincula directamente los elementos declarativos de la interfaz gráfica descritos en archivos XML con los atributos privados y métodos de eventos en los controladores Java, aislando la presentación de la lógica de control.

Manejo de Colecciones Dinámicas y Matrices Dinámicas: Estructuras de datos bidimensionales de memoria (Casilla[][]) unidas a colecciones ordenadas (List, ArrayList) aprovechadas para gestionar los algoritmos del núcleo del juego, tales como la distribución aleatoria de minas, el cálculo matemático de vecindades y el descubrimiento recursivo en cascada.

Expresiones Regulares (RegEx) y Sanitización de Inputs: Patrones de reconocimiento de cadenas de texto implementados en los controladores de registro para validar sintácticamente los formatos de datos de entrada (como la estructura obligatoria de un correo electrónico o la robustez de una contraseña) antes de su procesamiento o almacenamiento en los sistemas de persistencia.

Patrón Data Transfer Object (DTO): Estructura de diseño software implementada en el sistema (ViciadoDTO) para empaquetar, transformar y transportar datos agregados desde las consultas complejas de la base de datos hacia la interfaz del usuario, optimizando el rendimiento al evitar la carga completa de entidades pesadas.

Modularización de Plataforma (Java Platform Module System): Mecanismo de encapsulación de software estructurado en el archivo descriptor del sistema (module-info.java), utilizado para declarar explícitamente qué paquetes del proyecto se exponen y

qué módulos o librerías externas (como `javafx.controls`, `javafx.media`, `org.hibernate.orm.core` o `java.sql`) requiere obligatoriamente la aplicación para compilar y ejecutarse con seguridad.

Enumeraciones de Estado Fuertemente Tipadas (Enums): Clases especiales de datos (Nivel, View) empleadas a lo largo de toda la arquitectura para restringir y gobernar los estados válidos de la lógica de negocio del juego de forma estricta, previniendo errores de consistencia en tiempo de compilación al definir un conjunto cerrado de niveles de dificultad o rutas de pantallas.

Patrón de Arquitectura en Capas y Desacoplamiento DAO: Estructura organizativa aplicada al diseño de la persistencia que separa las interfaces de contrato (UsuarioDAO, PartidaDAO) de sus implementaciones tecnológicas concretas (UsuarioDAOImpl, PartidaDAOImpl), logrando que la lógica de juego sea agnóstica e independiente de la base de datos subyacentes.

Flujo de Entrada/Salida de Archivos (I/O Streams & Buffers): Mecanismos de comunicación con el sistema de almacenamiento físico local integrados a través de componentes como `BufferedReader` e `InputStream`, aprovechados para canalizar de forma eficiente y segura la carga de flujos de bytes desde los recursos internos embebidos en el empaquetado del software (como imágenes de bandera y archivos JSON).

Arquitectura de Software Guiada por Eventos (Event-Driven Programming): Paradigma de desarrollo sobre el que descansa toda la interacción de la interfaz gráfica a través de manejadores como `ActionEvent` y capturadores de periféricos físicos (como `MouseButton.PRIMARY` y `KeyCode`), lo que permite al sistema permanecer en escucha asíncrona permanente y reaccionar de manera inmediata a las pulsaciones de teclado y ratón del jugador.

Componentes de Analítica Visual (Charts & Progress Indicators): Elementos gráficos interactivos integrados en la interfaz de usuario (como `PieChart` y `ProgressBar`) para modelar visualmente datos analíticos agregados complejos (distribución de cuotas de niveles de juego y porcentajes de logros desbloqueados), facilitando la interpretación inmediata de métricas de rendimiento sin obligar al usuario a interpretar registros tabulares crudos.

Componentes de Control de Estado de Flujo (Pause & Resume Operations): Funcionalidad lógica y de interfaz diseñada para congelar instantáneamente los subprocesos de hilos de tiempo activos del juego (`TimeLine`, `AnimationTimer`), ocultando dinámicamente la matriz de datos del tablero a través de opacidades y deshabilitación de periféricos, salvaguardando la integridad de los récords frente a interrupciones externas.

Reinicio Dinámico de Ciclo de Vida (Soft Reset Action): Operación de refresco en caliente orientada al vaciado completo del contenedor visual (`GridPane.getChildren().clear()`) y la regeneración matricial de casillas en memoria, lo que permite al jugador inicializar una nueva sesión de juego de manera fluida y con latencia cero, reutilizando la instancia del controlador sin necesidad de recargar el árbol de nodos FXML desde el disco.

Gestión de Operaciones CRUD en Administración: Capacidad funcional integrada exclusivamente en la vista del Administrador Maestro para interactuar de forma directa

sobre el ciclo de vida de los registros de persistencia de usuarios (Lectura/Visualización de perfiles estadísticos y Eliminación o Borrado de cuentas), implementando mecanismos de control e integridad referencial en cascada sobre las bases de datos para salvaguardar el histórico global del sistema.

Despliegue y Alojamiento de Documentación Estática (GitHub Pages): Servicio en la nube utilizado para el despliegue e indexación del manual técnico y la guía de usuario web (index.html) mediante un entorno de producción accesible a través de protocolos web estándar (HTTP/HTTPS), facilitando la distribución autónoma de la documentación del software sin requerir almacenamiento en la máquina local del cliente.

Notificación Volátil de Feedback de la UI (Toast Notifications): Ventanas emergentes temporales y traslúcidas (Toast) autogestionadas mediante hilos asíncronos concurrentes para inyectar avisos informativos discretos en pantalla (como alertas de guardado o errores de input) que desaparecen gradualmente sin requerir una acción de cierre explícita del usuario, minimizando la fricción en la experiencia de navegación.

Ventanas Emergentes de Confirmación de Flujo (Modal Alert Blocks): Contenedores gráficos flotantes configurados para interceptar la ejecución de hilos de navegación críticos (como abandonar una partida en curso o eliminar un registro), bloqueando la interacción perimetral del sistema y forzando una selección explícita del usuario mediante botones tipados (ButtonType) para blindar la aplicación contra acciones accidentales destructivas.

Algoritmo de Despeje en Cascada (Flood Fill / Cascada Recursiva): Lógica algorítmica iterativa y recursiva de propagación por vecindad implementada en el método principal de apertura del tablero (revelarCasilla). Este algoritmo analiza matemáticamente las matrices bidimensionales de memoria en el eje de coordenadas (X, Y) para descubrir de forma automatizada e instantánea todas las casillas vacías contiguas, deteniendo la recursión exclusivamente al colisionar con casillas con valores numéricos limítrofes a minas.

Inyección de Componentes de Incremento Secuencial (Spinner Controls): Elementos de captura de datos en la interfaz gráfica (Spinner<Integer>) implementados en el menú de personalización para restringir las entradas del usuario a rangos numéricos discretos y estrictos mediante factores de valor (SpinnerValueFactory), impidiendo la inserción sintáctica de dimensiones inválidas o corrupciones de desbordamiento en memoria.

Sincronización de Propiedades Reactivas (Property Listeners): Mecanismo de escucha y enlace de cambios de estado (valueProperty().addListener) provisto por la arquitectura de JavaFX para capturar en tiempo de ejecución las variaciones en los inputs del usuario, permitiendo a la aplicación recalcular matemáticamente y actualizar dinámicamente en pantalla las sugerencias y los límites de seguridad del tablero (como actualizar el tope de minas al cambiar filas o columnas) con latencia cero.

Componentes de Información Flotante Contextual (Tooltip UI Elements): Ventanas de diálogo flotantes, autogestionadas y volátiles (Tooltip) vinculadas a los controles del menú (btnRanking, btnInfoContra) configuradas con retardos y duraciones explícitas de renderizado (setShowDelay, setShowDuration) para proveer al usuario de asistencia e instrucciones del juego inmediatas sin sobrecargar la composición visual permanente de la pantalla.

Mecánica Algorítmica de Despeje Acelerado (Chording): Operación lógica avanzada del motor de juego (ejecutarChording) vinculada a los controladores analógicos de los periféricos del ratón, diseñada para escanear en tiempo real las casillas adyacentes a una celda ya revelada; si el número de banderas marcadas en la vecindad matemática coincide con el valor numérico de la casilla central, el sistema desencadena de forma automatizada la apertura en bloque de todas las celdas limpias periféricas, acelerando el flujo de la partida.

Modificación de Tono Acústico en Caliente (Pitch Shifting): Técnica de procesamiento de señales multimedia integrada en el bucle del temporizador (AnimationTimer) que altera dinámicamente la frecuencia de muestreo y la velocidad de reproducción de un flujo de audio (soundAlert.setRate()) para volverlo significativamente más agudo y acelerado durante los últimos segundos del modo contrarreloj, proveyendo un estímulo auditivo de tensión psicológica que altera la experiencia inmersiva del usuario.

Sumario

Resumen.....	3
Abstract.....	4
Palabras Clave.....	5
Introducción/motivación.....	11
Objetivos.....	12
Estado del arte.....	13
Caso de estudio.....	14
Diagramas.....	15
Desarrollo del proyecto.....	16
Manual del Administrador.....	17
Manual del Usuario.....	18
Forma jurídica y trámites de constitución.....	19
Viabilidad tecno-económica.....	20
Trabajo futuro.....	21
Conclusiones.....	22
Biblioteca de recursos web y referencias.....	23
Anexos.....	24

• Introducción/motivación

El desarrollo de software enfocado al entretenimiento interactivo y de lógica representa uno de los desafíos más complejos para un programador, puesto que demanda una sincronización perfecta entre las estructuras de datos que gobiernan las reglas de negocio en el backend y los hilos de renderizado gráfico que gestionan la experiencia del usuario en el frontend. El presente proyecto nace con el propósito de diseñar, implementar y desplegar “MineManager Pro”, un sistema integral de escritorio estructurado bajo el patrón de diseño Modelo-Vista-Controlador (MVC) y fundamentado en las reglas clásicas del videojuego Buscaminas.

La elección de esta temática responde a factores tanto de índole técnica como de motivación personal del desarrollador. En primer lugar, existe una profunda afinidad y conocimiento empírico personal hacia las mecánicas y estrategias que rigen el Buscaminas tradicional. Este dominio absoluto del juego ha provisto una ventaja analítica crucial durante la fase de ingeniería, permitiendo trasladar de forma precisa algoritmos abstractos de matrices bidimensionales a código Java fuertemente tipado. El conocimiento profundo e intuitivo de cómo se calculan matemáticamente las vecindades de celdas, las condiciones inmediatas de victoria (al despejar con éxito todas las casillas libres de minas) y los disparadores de derrota (al interactuar con celdas reactivas explosivas), ha asegurado que el núcleo lógico del mapa (Tablero y Casilla) emule con absoluta fidelidad el comportamiento del software original.

Desde la perspectiva puramente académica y formativa, el Buscaminas clásico provee el escenario perfecto para actuar como catalizador de los conocimientos avanzados adquiridos en los diferentes módulos didácticos. Aunque el planteamiento jugable es de asimilación inmediata, su trasfondo técnico es de una complejidad matemática y arquitectónica sumamente rica:

1. Estructuras de Datos Avanzadas: Requiere el gobierno de colecciones y arrays dinámicos bidimensionales en memoria, aplicando conceptos puros de programación orientada a objetos (POO) sobre cada casilla del tablero.
2. Desafíos Algorítmicos: Demanda la implementación de algoritmos recursivos complejos, como la cascada de descubrimiento por inundación (Flood Fill) al pulsar casillas vacías, o la optimización de interacciones del usuario en periféricos físicos como el Chording.
3. Persistencia y Entorno Empresarial: Permite evolucionar un juego tradicional en una suite corporativa moderna mediante la incorporación de un sistema de control de accesos basado en roles de seguridad (RBAC) gobernado por un ORM robusto como Hibernate unida a una base de datos MySQL, sumando además auditorías de administración, exportaciones paralelas a sistemas de archivos (CSV) e interconexiones de red asíncronas para la mensajería automatizada por correo electrónico.

En conclusión, “MineManager Pro” no se plantea como un simple clon recreativo, sino como un proyecto de ingeniería de software maduro que unifica la pasión y el entendimiento profundo de una lógica de juego con las prácticas más rigurosas de desarrollo de interfaces enriquecidas, seguridad, persistencia distribuida y optimización de

hilos concurrentes exigidas en el entorno profesional contemporáneo.

• Objetivos

El propósito central de este proyecto es el diseño y desarrollo de una aplicación de escritorio sólida y madura que evolucione el clásico juego del Buscaminas hacia una suite informática segura, escalable y auditable, unificando el entretenimiento lógico con prácticas avanzadas de desarrollo corporativo.

Objetivos Generales:

Desarrollo Integral Multiplataforma: Diseñar una aplicación de escritorio orientada al entretenimiento y la analítica visual, integrando una interfaz gráfica rica y reactiva con un motor de persistencia relacional robusto.

Consolidación Académica: Aplicar de forma práctica las competencias de ingeniería del software adquiridas a lo largo del Ciclo.

Solución Profesional Autónomo: Compilar un software autónomo (standalone) con un acabado robusto que demuestre la viabilidad de acoplar algoritmos lógicos con estrictos requisitos de seguridad y administración.

Objetivos Específicos Técnicos:

Modularización y Arquitectura Desacoplada: Estructurar el código aislando las responsabilidades mediante el patrón MVC y el sistema de módulos de Java, garantizando que la lógica de juego sea independiente de las vistas declarativas y de la persistencia.

Persistencia Avanzada e Híbrida (ORM): Configurar el framework Hibernate sobre MySQL para automatizar el ciclo de vida de los datos (Usuario, Partida, Logro), complementándolo con un gestor local de archivos de entrada/salida para respaldos en texto plano (CSV), y diccionarios de configuración dinámicos en formato JSON.

Optimización y Analítica Eficiente: Implementar el patrón DTO junto con proyecciones de la API Criteria para agilizar el procesamiento de consultas agregadas y estadísticas complejas, disminuyendo la latencia y la carga de memoria en la máquina cliente.

Concurrencia y Sincronización asíncrona: Diseñar flujos concurrentes temporizados en hilos independientes mediante Timeline y AnimationTimer para gobernar el cronómetro de precisión, los efectos de parpadeo y la renderización en cascada sin bloquear el hilo principal de la interfaz.

Criptografía y Servicios de Red: Proteger las credenciales de los usuarios en base de datos mediante técnicas de hashing unidireccional y conectar la lógica con redes externas a través de Java Mail API para permitir el despacho automatizado y seguro de correos SMTP de recuperación:

Objetivos Específicos Funcionales:

Experiencia Jugable Personalizable: Ofrecer tres dificultades preestablecidas y un menú de configuración personalizada controlado por componenets Spinner y escuchas reactivas (PropertyListeners) para validar las dimensiones del tablero y el volumen de minas en tiempo real.

Mecánicas Clásicas y Alternativas Avanzadas: Proveer al jugador de una interacción

fluida basada en algoritmos recursivos de descubrimiento por vecindad y atajos físicos (Chording), junto a un Modo Contrarreloj enriquecido con dinámicos de tensión psicológica mediante modificaciones acústicas (pitch shifting).

Incentivos y Recompensas Automatizadas: Desarrollar un servicio dinámico que evalúe el rendimiento del usuario tras cada sesión, desbloqueando insignias visuales de manera asíncrona en su vitrina de logros.

Control de Accesos Basado en Roles: Restringir los privilegios del sistema dividiendo el flujo en dos perfiles: el rol Jugador, con acceso exclusivo a analíticas personales mapeadas en PieChart y ProgressBar, y el rol Administrado Maestro, facultado para auditar accesos globales y ejecutar operaciones CRUD destructivas sobre las cuentas.

Control de Interacción y Usabilidad: Mitigar la fricción en la experiencia de usuario proveyendo conmutadores de audio (Mute), reinicios fluidos de partida, avisos volátiles integrados e interceptores modales que blinden el sistema frente a acciones accidentales.

• Estado del arte

El Buscaminas es un videojuego clásico de lógica cuyas primeras versiones se popularizaron a nivel global en la década de 1990 al venir integrado en el sistema operativo Microsoft Windows. A día de hoy, el mercado ofrece diversas soluciones basadas en este concepto, las cuales se analizan a continuación para contrastar sus características con la propuesta desarrollada en este proyecto.

Análisis de Aplicaciones Similares en el Mercado

Buscaminas Clásico de Microsoft (Minesweeper de Windows):

Puntos Fuertes: Excelente optimización a nivel de sistema operativo, jugabilidad pulida y fluida, e integración de mecánicas estándar como el Chording clásico de forma nativa.

Puntos Débiles: Carece de un sistema de persistencia relacional avanzado para la gestión multiusuario local. No implementa un control de accesos por seguridad ni módulos de auditoría administrativa, limitándose a guardar marcas de tiempos locales en registros planos del sistema.

Versiones Web y Clones Open Source (ej. Minesweeper.online):

Puntos Fuertes: Accesibilidad inmediata desde cualquier navegador web y rankings globales competitivos basados en arquitecturas de red servidor.

Puntos Débiles: Alta dependencia de la conectividad a Internet. Desde el punto de vista del software de escritorio, estas aplicaciones suelen presentar una experiencia de usuario (UX) ralentizada por el renderizado de servicios de escritorio.

Ventaja Diferencial de MineManager Pro:

Frente a las alternativas del mercado, MineManager Pro destaca por transformar un entorno recreativo en una herramienta híbrida de gestión y analítica de escritorio. Sus ventajas clave radican en la implementación de una seguridad basada en roles, un panel de administración avanzado para la auditoría y gestión CRUD de usuarios, analíticas visuales integradas y una persistencia local redundante (MySQL con respaldos en CSV e inicialización por JSON), todo ello enriquecido con un modo contrarreloj reactivo multimedia.

Tecnologías Utilizadas y Alternativas Técnicas

Desarrollo de Interfaces (JavaFX frente a Swing/AWT): Se seleccionó JavaFX debido a su arquitectura moderna orientada a la aceleración por hardware y su compatibilidad con el patrón MVC mediante archivos declarativos FXML y estilos CSS. La alternativa tradicional, Java Swing, se descartó por considerarse obsoleta, carecer de soporte nativo para efectos polifónicos dinámicos complejos y ofrecer componentes visuales rígidos y de costosa personalización.

Capa de Persistencia (Hibernate/MySQL frente a JDBC Nativo): La utilización del ORM Hibernate permite gestionar los datos de forma orientada a objetos abstractos, garantizando la portabilidad del código y la seguridad frente a inyecciones SQL mediante consultas tipadas de la API Criteria. La alternativa de usar JDBC Nativo con consultas escritas a mano en texto plano se rechazó debido a que eleva exponencialmente las

líneas de código propensas a errores de sintaxis y acopla de manera rígida el software a un motor de base de datos específico.

Intercambio y Respallos (CSV/JSON frente a XML): Para la persistencia híbrida se optó por CSV (portabilidad analítica plana) y JSON (configuración dinámica ágil). Se evaluó el uso de XML, pero fue desestimado por su excesiva verbosidad y el alto consumo de recursos de cómputo que requieren sus componentes de procesamiento (parsers DOM/SAX) en entornos de escritorio livianos.

• Caso de estudio

El desarrollo de “MineManager Pro” aborda la transformación de una mecánica lógica tradicional en una herramienta de escritorio segura, controlada y enriquecida con análisis estadísticos. En este apartado se exponen las especificaciones funcionales y de diseño algorítmico aplicadas para solucionar las carencias del software analizado en el Estado del Arte.

Ámbito y Reglas del Núcleo del Juego

El sistema gobierna un tablero matricial bidimensional dinámico compuesto por objetos Casilla. La lógica del Game Loop se rige por las siguientes especificaciones estrictas:

Garantía del Primer Clic: El mapa se genera en memoria de forma transparente al usuario, pero la distribución aleatoria de las minas en la matriz (`tablero.colocarMinas`) no se ejecuta hasta que el jugador interactúa con la primera casilla. Esto blindará el flujo de juego, garantizando matemáticamente que la primera pulsación del usuario sea siempre segura y libre de explosivos.

Cálculo de Proximidad y Despeje: Cada celda libre calcula en tiempo de compilación el número de minas limítrofes. Al interactuar con una casilla de valor cero, se invoca un algoritmo de inundación recursiva en cascada que despeja de forma instantánea el perímetro seguro, deteniéndose al colisionar con celdas numéricas para ofrecer una respuesta de juego fluida.

Condiciones Dinámicas de Finalización: El sistema evalúa tras cada clic los disparadores de estado. La victoria se computa cuando el número de casillas ocultas coincide exactamente con el volumen de minas configurado. La derrota se activa de inmediato al revelar una mina culposa, deteniendo los temporizadores concurrentes y ejecutando un bucle de animación cronometrada (`revelarMinasAnimado`) para mostrar el resto de explosivos en el tablero.

Restricciones del Sistema y Validación

Para salvaguardar la estabilidad de la aplicación frente a desbordamientos de memoria (`BufferOverflow`) o inputs inválidos por parte del usuario, se implementan las siguientes restricciones en la capa de control:

Sustentación de Dimensiones del Tablero: En el módulo de personalización, los Spinners acotan los rangos estrictos (filas entre 5 y 30; columnas entre 5 y 50). El editor de texto se sanitiza mediante expresiones regulares (`TextFormatter` con patrones `\\d*`), bloqueando la inserción de caracteres alfabéticos o símbolos.

Límites Matemáticos de Seguridad: La proporción de minas se calcula dinámicamente en base a las propiedades reactivas del tamaño del mapa. El sistema restringe el número de minas a un mínimo del 2% (para asegurar el reto lógico) y a un máximo del 50% del total de casillas del tablero (evitando la saturación física del espacio y el bloqueo de los algoritmos de despeje).

Prevención de Incompatibilidad de Resolución: Si el usuario parametriza dimensiones que exceden el tamaño estándar del monitor (más de 20 columnas o 15 filas), el sistema intercepta el flujo activando un aviso preventivo visual en pantalla (`paneAvisoResolucion`).

y escala proporcionalmente las dimensiones físicas de los botones a un tamaño menor para garantizar la correcta visualización del Grid sin desbordar los márgenes de la ventana.

Flujo de Navegación y Control de Estados de Sesión

La arquitectura del núcleo (AppShell) centraliza el intercambio de pantallas mediante el patrón Singleton, gestionando los flujos a través de las siguientes directrices funcionales:

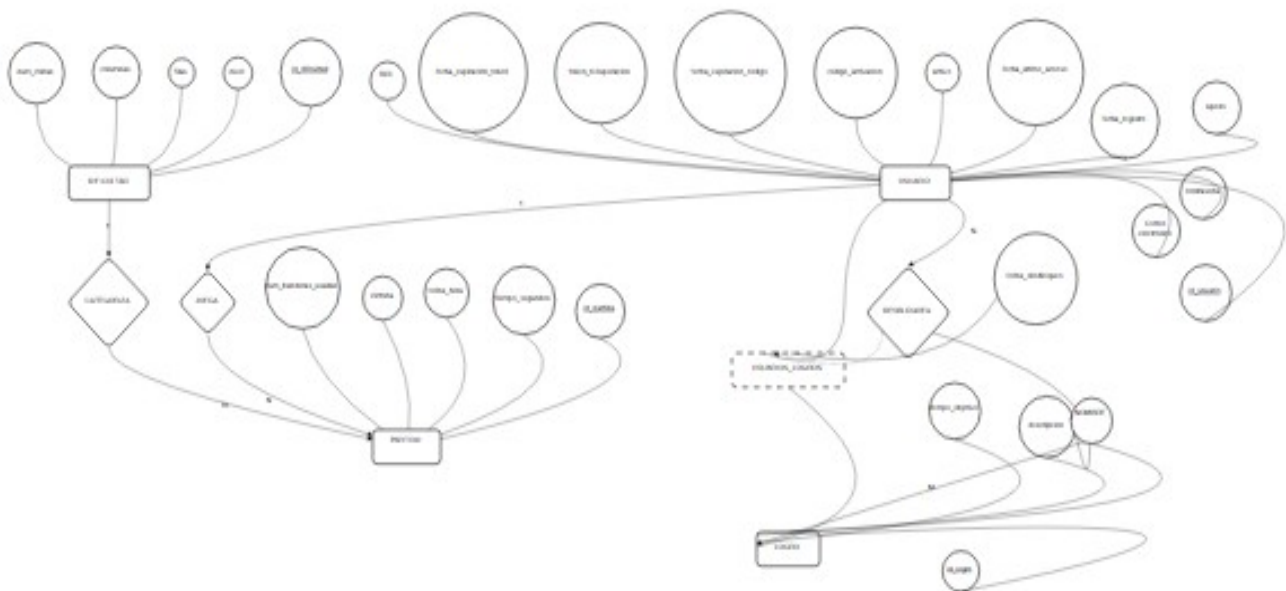
Control de Interrupciones (Pausa): Al conmutar el estado de pausa, se congelan asíncronamente los contadores de tiempo (Timeline y AnimationTimer) y se oculta el tablero de juego sustituyéndolo por un panel opaco. Esto neutraliza la posibilidad de fraude o trampas, impidiendo que el jugador analice la matriz con el reloj detenido.

Protección de Navegación Crítica: Los hilos de salida accidentales hacia el menú o cierres de sesión en mitad de una partida en curso se interceptan mediante bloques de diálogo modales (paneConfirmacionAbandono), forzando al usuario a confirmar explícitamente la acción y pausando temporalmente el entorno.

Auditoría Administrativa: El sistema bifurca las vistas en el arranque en base al rol autenticado. Un usuario estándar accede exclusivamente a su vitrina de logros y analíticas personales de rendimiento, mientras que las credenciales de administración habilitan módulos avanzados para la gestión de cuentas e inspección de estadísticas agregadas.

- **Diagramas**

E-R:



Para la persistencia de datos en *MineManager FX*, he diseñado un modelo relacional centrado en la interacción del usuario con las mecánicas del juego y su progresión. Las entidades principales son las siguientes:

- **USUARIO:** Es la entidad base del sistema. Contiene los datos de autenticación (nickname, password cifrada, email) y control de acceso (token_recuperacion, rol). Es el eje central ya que está vinculada a las partidas jugadas y a los logros alcanzados.
- **DIFICULTAD:** Entidad de configuración que define los parámetros del tablero (filas, columnas, num_minas) y el nivel (FACIL, MEDIO, etc.). Esta tabla permite que el sistema sea dinámico sin alterar el código fuente del juego.
- **PARTIDA:** Entidad transaccional que registra cada juego. Mantiene una relación de **1:N** con USUARIO (un usuario tiene muchas partidas) y **1:N** con DIFICULTAD (una configuración de dificultad se usa en muchas partidas). Registra el tiempo_segundos, fecha_hora y victoria (booleano).
- **LOGRO:** Representa los objetivos desbloqueables del juego.
- **USUARIOS_LOGROS:** Actúa como tabla asociativa para resolver la relación **M:N** entre USUARIO y LOGRO, almacenando la fecha_desbloqueo del logro.

Relaciones Clave:

1. **JUEGA (1:N):** Define que un usuario puede participar en múltiples partidas, pero cada partida pertenece a un único usuario (identificado por id_usuario).
2. **CATEGORIZA (1:N):** Vincula cada partida con su nivel de dificultad. Esto permite realizar consultas estadísticas para comparar tiempos según el nivel jugado.
3. **DESBLOQUEA (M:N):** Relaciona usuarios y logros. Un usuario puede desbloquear varios logros, y un logro puede ser obtenido por múltiples usuarios.

Diagramas de clases

Controllers:

Este diagrama de clases muestra la estructura de la capa de controladores de la aplicación, detallando los atributos y métodos que gestionan la interfaz de usuario, la navegación entre pantallas y la lógica de interacción en los distintos modos de juego.

La arquitectura se organiza bajo el patrón MVC, donde los controladores actúan como intermediarios entre las vistas FXML y los modelos de datos. Destaca especialmente el GameController, que encapsula la lógica algorítmica del Buscaminas (gestión de minas, cronómetros, efectos sonoros y estados de victoria), y el AdminController y RankingController, que gestionan las operaciones CRUD y el filtrado avanzado de datos mediante el uso de listas observables y filtros dinámicos.

DAOS & DTO:

Este diagrama de clases detalla la capa de persistencia de la aplicación. Su función es abstraer la complejidad de las operaciones de base de datos, separando la lógica de negocio de la implementación técnica de persistencia mediante el framework Hibernate.

Descripción de los componentes:

- **Interfaces DAO (LogroDAO, PartidaDAO, UsuarioDAO):** Definen el contrato de operaciones necesarias (CRUD) para gestionar las entidades. Esta capa de abstracción facilita el mantenimiento y la escalabilidad del sistema, permitiendo cambiar la implementación de base de datos sin afectar a la lógica de negocio.
- **Implementaciones (DAOImpl):** Clases que ejecutan las consultas HQL (Hibernate Query Language). Se ha priorizado el uso de JOIN FETCH para optimizar el rendimiento de las consultas y minimizar el número de peticiones a la base de datos (evitando el problema de *N+1 queries*).
- **Patrón DTO (ViciadoDTO):** Se ha implementado el patrón *Data Transfer Object* para consolidar datos procedentes de consultas complejas (como la suma de tiempos jugados por usuario) en un objeto simple, diseñado específicamente para ser visualizado en la capa de presentación (RankingController) sin exponer la entidad de dominio completa.

MODEL:

Esta capa contiene la representación de la lógica de negocio y las entidades mapeadas con la base de datos relacional. Se ha diseñado siguiendo los principios de la Programación Orientada a Objetos para garantizar la cohesión y el encapsulamiento.

Capa de Servicio y Lógica de Negocio

Este diagrama representa la capa de servicios, encargada de centralizar la lógica de negocio de la aplicación. Las clases principales, LogroService y EmailService, orquestan las reglas de gamificación y la comunicación externa interactuando directamente con los DAOs. Este diseño permite mantener la lógica de las recompensas y las notificaciones por correo electrónico aislada de la interfaz gráfica, facilitando su reutilización y pruebas unitarias independientes.

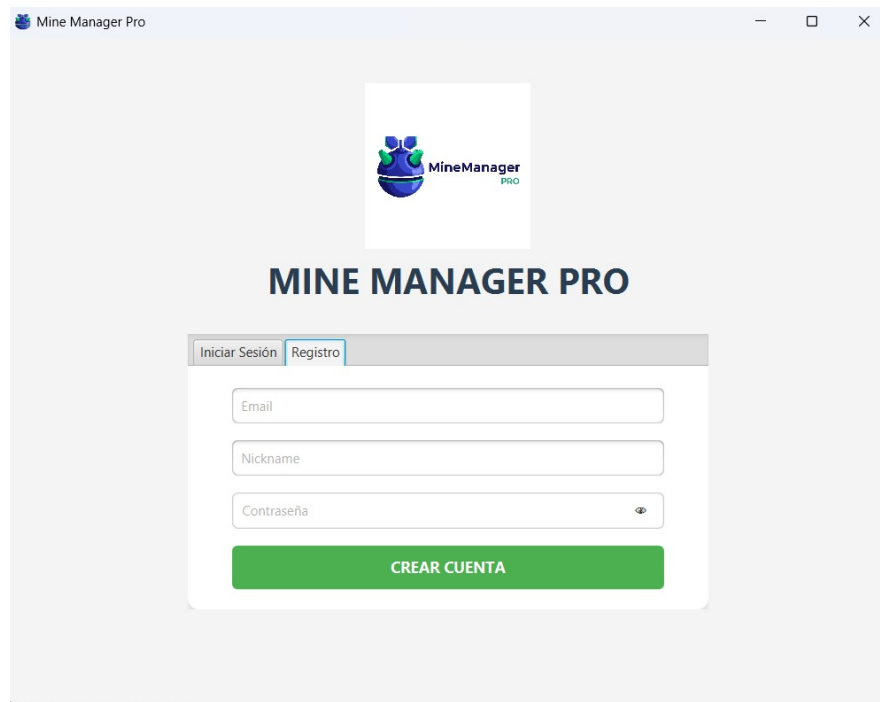
Capa de Infraestructura y Utilidades

Este diagrama representa la capa de utilidades e infraestructura, encargada de proporcionar soporte técnico transversal a toda la aplicación. Agrupa herramientas independientes de la lógica de negocio, como la gestión de persistencia (HibernateUtil), el control de la navegación y estado global de la sesión (AppShell), el filtrado de contenidos inapropiados (FiltroNombre), la gestión de archivos (CSVManager) y servicios de interfaz de usuario como las notificaciones no bloqueantes (Toast).

• Desarrollo del proyecto

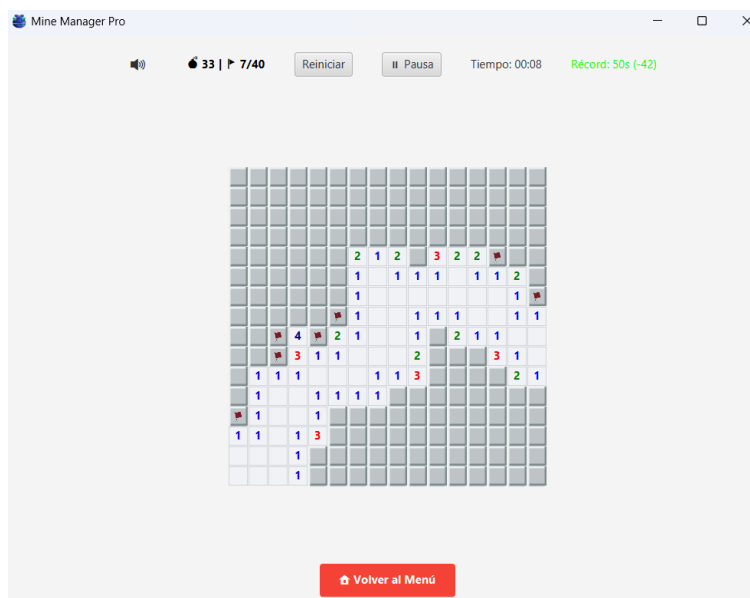
El desarrollo de *MineManager Pro* se ha basado en un patrón **MVC (Modelo-Vista-Controlador)**. La persistencia se gestiona mediante **Hibernate 6**, lo que permite una abstracción total frente a MySQL, mientras que la interfaz gráfica ha sido construida utilizando **JavaFX con FXML**, permitiendo una separación clara entre la estructura visual y la lógica de control.

LOGIN



Esta ventana implementa la autenticación mediante **BCrypt**. El usuario es validado contra la base de datos usuarios. Se han incluido validaciones de campos vacíos y protección contra inyección SQL mediante parámetros en las consultas (HQL).

PARTIDA



Se ha optimizado la interfaz para que las celdas sean adaptativas según la resolución, asegurando que niveles como el "Difícil" (16x30) sean jugables en cualquier monitor sin desbordar el contenedor.

PANEL DE ADMIN



Este panel es el eje de la auditoría. Permite al Administrador Maestro cambiar roles (USER a ADMIN) mediante el método `cambiarRol` del `UsuarioDAO`. La tabla está vinculada a un `FilteredList` que permite buscar usuarios en tiempo real, facilitando la gestión de grandes volúmenes de datos.

CASOS DE PRUEBA

ID Caso	Funcionalidad	Descripción del Test	Resultado Esperado
CP-01	Login / Autenticación	Introducir credenciales incorrectas.	El sistema deniega el acceso y muestra un Toast de error.
CP-02	Seguridad (RBAC)	Acceso de usuario USER a panel Admin.	La interfaz oculta el botón de Admin y deniega acceso por URL.
CP-03	Lógica de Juego	Revelar una mina  en nivel Fácil.	El juego detona, pausa el cronómetro y muestra mensaje de derrota.
CP-04	Persistencia	Completar nivel y comprobar BBDD.	El registro de la partida queda guardado en la tabla partidas.
CP-05	Validación CSV	Importar archivo de ranking corrupto.	El CSVManager lanza excepción controlada y muestra alerta visual.

• Manual del Administrador

El sistema de gestión de *MineManager Pro* está diseñado para ofrecer al administrador un control absoluto sin que sea necesario un conocimiento profundo de la arquitectura interna del software. El despliegue inicial es sencillo, requiriendo únicamente la ejecución del script SQL suministrado en el repositorio, el cual configura automáticamente el esquema relacional, las dificultades predefinidas y el catálogo de logros.

Una vez desplegada la base de datos, la configuración del entorno se centraliza en un único archivo de propiedades externo. Esta externalización es crítica, ya que permite al administrador ajustar los parámetros de conexión, las credenciales del servidor SMTP para el envío de notificaciones y otros valores de configuración sin tener que recompilar el código fuente. Para la operativa diaria, el administrador dispone de un panel exclusivo accesible únicamente tras la validación de sus credenciales, que le permite filtrar el histórico de usuarios mediante herramientas de búsqueda en tiempo real, modificar privilegios mediante la alternancia de roles y auditar el histórico de partidas.

En caso de incidencias, el sistema está preparado para la monitorización básica a través de la consola de errores, donde se registran las excepciones capturadas en la capa de servicios. Este diseño permite que cualquier error en la capa de persistencia o en la conexión de red quede debidamente documentado, permitiendo al administrador tomar medidas correctoras rápidas sin comprometer la disponibilidad del juego. La integridad de este manual y la sencillez de los procedimientos descritos garantizan que el mantenimiento de la plataforma pueda ser llevado a cabo de forma eficiente y segura.

• Manual del Usuario

Pantalla de Autenticación y Registro

Usuario	Tiempo (s)	Fecha
antonfaraldo	6	31/05/2026 18:25
admin	9	31/05/2026 15:00
antonfaraldo	10	31/05/2026 18:26
antonfaraldo	11	31/05/2026 18:25
admin	12	31/05/2026 15:08

Esta interfaz integra tanto el inicio de sesión como el registro de nuevos usuarios, permitiendo alternar entre ambos formularios de manera ágil. Entre sus características destacadas se incluyen:

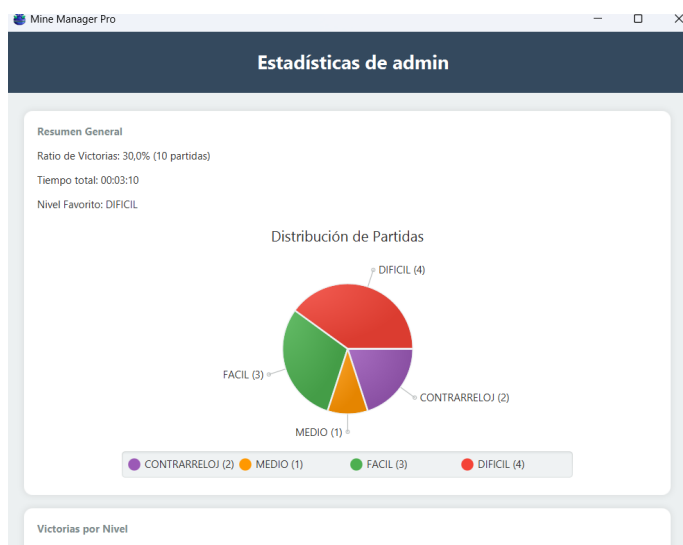
- **Atajos de Teclado:** Se ha optimizado la experiencia mediante el uso de la tecla Enter para activar el botón de acción principal (Login o Registro), permitiendo al usuario completar el proceso sin necesidad de usar el ratón.
- **Visibilidad de Contraseña:** Se ha incluido un icono de "ojo" que permite al usuario alternar entre ocultar y mostrar el contenido del campo de contraseña, facilitando la detección de errores de escritura.
- **Aviso de Mayúsculas:** Ante el riesgo de errores por tener activado el bloqueo de mayúsculas (*Caps Lock*), la vista muestra un mensaje de advertencia dinámico,

evitando frustraciones innecesarias durante el acceso.

- Validación y Limpieza: El sistema realiza una limpieza automática de espacios en blanco (*trimming*) en los campos de usuario y correo para evitar errores de persistencia.
- Recuperación de Cuenta: En caso de olvido de contraseña, el enlace "Recuperar contraseña" redirige al usuario a un flujo automatizado. El sistema genera un token único que es enviado al correo electrónico registrado, el cual debe ser validado antes de permitir el restablecimiento de las credenciales.

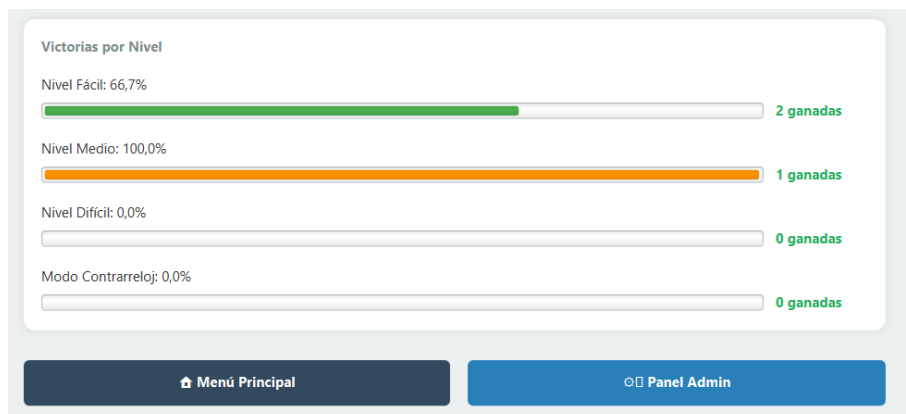
MENÚ PRINCIPAL

Esta vista
Desde
iniciar una
un nivel
ranking
logros
detecta
usuario
privilegios de administrador para mostrar, en caso afirmativo, el acceso al Panel de Control.



actúa como *hub* principal. aquí, el usuario puede partida rápida, configurar personalizado, acceder al global o consultar sus obtenidos. El sistema automáticamente si el autenticado posee

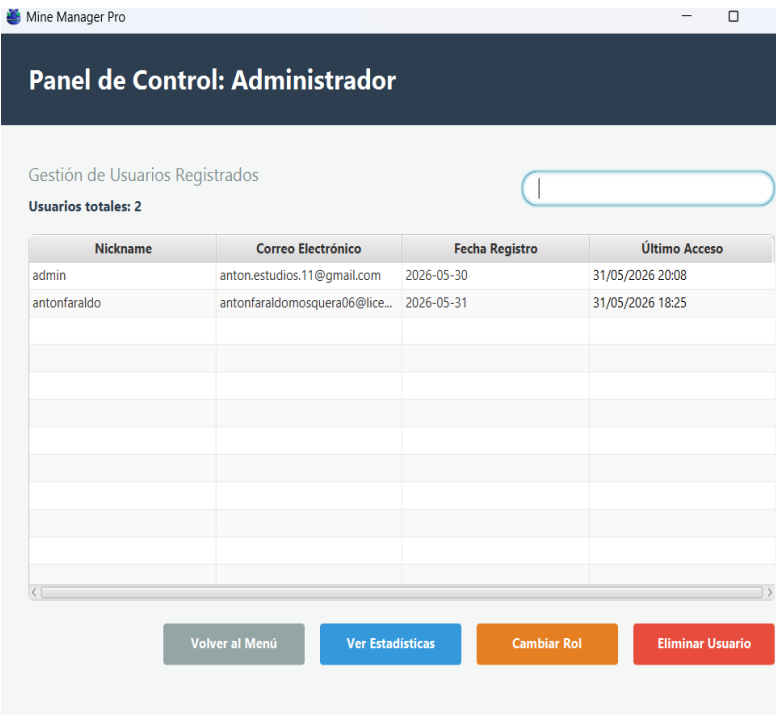
JUEGO



MÓDULO DE

Es la vista central de la aplicación, donde reside el motor lógico del Buscaminas. La interfaz ha sido diseñada para equilibrar el desafío técnico con una experiencia de usuario (UX) rica y reactiva.

- Mecánicas de Interacción:
 - Revelado y Banderas: El tablero se genera dinámicamente mediante el algoritmo de *Flood Fill*. El usuario utiliza el clic izquierdo para revelar celdas y el clic derecho para gestionar banderas.
 - Chording: Se ha implementado la mecánica de *Chording* (doble clic sobre una celda revelada), permitiendo agilizar el juego al descubrir automáticamente las celdas adyacentes seguras.
- Gestión de Partida:
 - Sistema de Control: Incluye un contador de banderas y bombas en tiempo real. Si el jugador coloca un número de banderas superior al de bombas disponibles, el contador se torna rojo, alertando visualmente al usuario sobre un posible error lógico.
 - Pausa y Cronómetro: El cronómetro dinámico integra bonificaciones de tiempo en modos contrarreloj. El botón de pausa oculta el tablero — preservando la integridad de la partida durante ausencias— y detiene el



- contador de tiempo.
- Navegación Segura: Para evitar la pérdida accidental de progreso, el botón de "Volver al Menú" activa un diálogo de confirmación que solicita al usuario validar su intención antes de abandonar la partida.
- Feedback y

Multimedia:

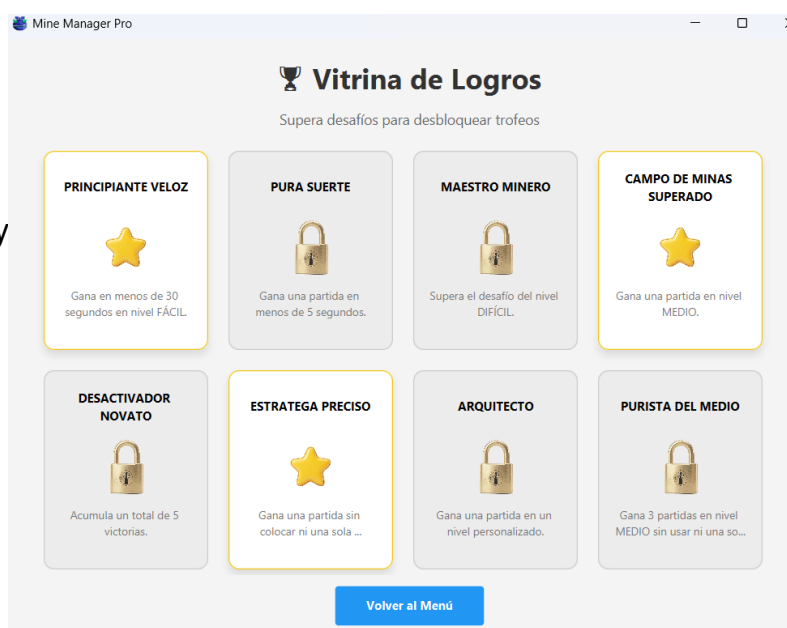
- Animaciones: Se ha integrado un efecto de confeti que se activa al lograr la victoria y una animación de escala (zoom) al activar una mina, ofreciendo un

feedback inmediato y claro del estado de la partida.

- **Configuración:** El usuario dispone de un control de audio accesible directamente desde la interfaz, permitiendo alternar el sonido del juego según sus preferencias.

RANKING

Esta interfaz permite al usuario consultar su rendimiento y compararse con la comunidad, integrando funcionalidades de filtrado y gestión de datos externos.



interfaz permite al usuario consultar su rendimiento y compararse con la comunidad, integrando funcionalidades de filtrado y gestión de datos externos.

Funcionamiento y Dinámica:

- **Filtrado por Dificultad:** La vista utiliza un sistema de pestañas (*TabPane*) que permite segmentar los resultados por dificultad (Fácil, Medio, Difícil, Contrarreloj) y una categoría especial denominada "Los más viciados".

- **Buscador y Filtros:** Dispone de un campo de búsqueda en tiempo real para filtrar usuarios específicos y un *ComboBox* para filtrar por tipo de partida. Además, incluye un *checkbox* de "Solo Top 10" para limitar la visualización a los mejores resultados, mejorando la legibilidad.
- **Gestión de Datos (Importación):** Se ha implementado un sistema de importación de rankings mediante archivos CSV. Esto permite que el usuario pueda cargar resultados externos o gestionar copias de seguridad de sus datos de competición.
- **Resaltado de Usuario:** Para mejorar la UX, las filas correspondientes al usuario autenticado actual resaltan con colores diferenciados, facilitando su localización dentro de la tabla.

ESTADISTICAS



Esta sección permite al usuario realizar un seguimiento detallado de su desempeño en la plataforma. La vista se ha diseñado para ofrecer una lectura rápida de los indicadores de rendimiento mediante una combinación de datos textuales y representaciones gráficas.

- **Resumen de Rendimiento:** La parte superior de la vista muestra indicadores clave de rendimiento (KPIs), tales como el **Ratio de Victorias** (calculado sobre el total de partidas jugadas), el **Tiempo total** invertido en la plataforma y la identificación del **Nivel Favorito** basado en la frecuencia de juego.
- **Análisis Gráfico:** La vista incorpora un *PieChart* interactivo que muestra la **Distribución de Partidas** por niveles (Fácil, Medio, Difícil y Contrarreloj). Esta herramienta permite al usuario comprender visualmente en qué modalidades invierte más tiempo.
- **Progresión por Niveles:** En la sección inferior, se muestran barras de progreso horizontales que detallan el porcentaje de éxito (victorias/derrotas) por cada nivel de dificultad, facilitando la identificación de áreas de mejora.
- **Lógica de Acceso Condicional:** Como medida de seguridad y segmentación de interfaz, el sistema integra una lógica de control de acceso:
 - **Visibilidad:** Aunque cualquier usuario registrado puede visualizar sus propias estadísticas, el botón **"Panel Admin"** y ciertas métricas globales solo se renderizan y habilitan si el sistema detecta que el usuario tiene el rol

de administrador.

- **Navegación:** Los botones inferiores permiten regresar al "Menú Principal" o acceder al "Panel Admin", este último solo visible tras la validación de credenciales con privilegios elevados.

PANEL ADMIN

La aplicación implementa una vista de gestión avanzada, destinada exclusivamente a usuarios con privilegios de administrador, facilitando el mantenimiento y control de la base de datos de usuarios de la plataforma

Esta interfaz centraliza la información mediante una tabla estructurada que permite realizar un seguimiento eficiente de los jugadores:

- **Visualización y Control:** La tabla presenta campos clave como Nickname, Correo Electrónico, Fecha de Registro y Último Acceso, permitiendo al administrador auditar la actividad de los usuarios.
- **Filtrado Dinámico:** Se ha integrado una barra de búsqueda superior que permite localizar usuarios de forma ágil mediante el filtrado por su Nickname.
- **Acciones Administrativas:** En la parte inferior, el panel incluye botones de acción rápida para gestionar el ciclo de vida de las cuentas:
 - **Ver Estadísticas:** Acceso directo a métricas de rendimiento del usuario seleccionado.
 - **Cambiar Rol:** Permite modificar los niveles de acceso (usuario vs. administrador) según las necesidades de la plataforma.
 - **Eliminar Usuario:** Mecanismo para dar de baja registros de la base de datos, garantizando la limpieza del sistema.

VITRINA DE LOGROS

Esta sección de la aplicación constituye el sistema de incentivos y gamificación de *MineManager FX*. Su objetivo es fomentar la fidelización del usuario mediante la consecución de hitos y desafíos específicos.

- **Interfaz de Desafíos:** La vista presenta una cuadrícula de tarjetas donde cada elemento representa un logro diferente.
- **Estado de los Logros:**
 - **Logros Desbloqueados:** Se visualizan con un icono distintivo (una estrella dorada) y resaltados con un borde destacado. Esto permite al usuario identificar rápidamente su progreso y nivel de pericia en el juego.
 - **Desafíos Pendientes:** Los logros aún no alcanzados se presentan mediante una interfaz con un icono de candado, indicando que el objetivo está bloqueado.
- **Descripción de los Desafíos:** Cada tarjeta incluye una breve descripción de las condiciones necesarias para desbloquear el logro (por ejemplo: "Gana una partida en menos de 30 segundos en nivel FÁCIL"). Esto guía al usuario hacia metas concretas, aumentando la rejugabilidad.
- **Navegación:** La vista cuenta con un botón inferior, "**Volver al Menú**", que garantiza una navegación intuitiva y coherente con el resto de las vistas de la aplicación.
- **Funcionalidad:** Esta vitrina se alimenta de la base de datos a través de la entidad LOGRO, utilizando un sistema de validación que consulta los registros de partidas del usuario para actualizar el estado de los logros en tiempo real.

PERSONALIZAR NIVEL

Esta vista permite al usuario romper con los estándares predefinidos del juego, ofreciendo una experiencia de *Buscaminas* adaptada a sus preferencias personales. La interfaz ha sido diseñada para garantizar la estabilidad del motor de juego mediante validaciones estrictas.

- **Parámetros de Configuración:**
 - **Dimensiones del Tablero:** El usuario puede definir el número de *Filas* y *Columnas*, permitiendo tableros de tamaño variable.
 - **Densidad de Minas:** El usuario puede seleccionar el número total de minas que se generarán aleatoriamente en el área de juego.
- **Mecanismos de Validación y Seguridad:**
 - **Límites de Seguridad:** Con el fin de evitar configuraciones imposibles o que comprometan el rendimiento, el sistema implementa validaciones en tiempo real que informan al usuario sobre los rangos permitidos (entre 5 y 30 filas, y entre 5 y 50 columnas).
 - **Lógica de Juego:** Se ha implementado un control de densidad de minas, limitando el número de estas entre un 2% y un 50% del total de celdas del tablero.
 - **Asistencia al Usuario:** La interfaz muestra dinámicamente un valor *Recomendado (estándar 20%)*, lo cual ayuda a jugadores menos experimentados a equilibrar la dificultad de forma sencilla.
- **Acciones:**
 - **Empezar Partida:** Valida los parámetros introducidos antes de instanciar el nuevo tablero.
 - **Cancelar:** Permite retornar al menú principal, abortando cualquier modificación.

Forma jurídica y trámites de constitución

Para la puesta en marcha y comercialización de *MineManager FX*, se ha determinado que la forma jurídica más adecuada es la del empresario individual, comúnmente conocida como autónomo. Esta elección responde a la naturaleza de un proyecto de desarrollo de software a pequeña escala, donde la actividad depende esencialmente de la labor creativa y técnica del promotor. Al tratarse de un modelo de negocio que no requiere una gran inversión inicial ni aportaciones de capital social obligatorio, la figura del autónomo permite una gestión administrativa mucho más ágil y eficiente, eliminando la carga burocrática que implicarían otras formas societarias.

En cuanto al nivel de responsabilidad y los riesgos asociados, si bien esta forma jurídica conlleva una responsabilidad ilimitada, el perfil de *MineManager FX* presenta un riesgo moderado. El desarrollo es propiedad intelectual del autor y los costes operativos son reducidos, por lo que la simplicidad en la toma de decisiones y la gestión fiscal compensan con creces el riesgo patrimonial. Esta estructura garantiza que la mayor parte de los recursos obtenidos se reinviertan directamente en el mantenimiento y evolución de la plataforma.

Respecto a los trámites para la constitución, el proceso se inicia con el alta censal en la Agencia Tributaria mediante la presentación de los modelos 036 o 037, donde se notifica el comienzo de la actividad y se selecciona el epígrafe correspondiente del Impuesto sobre Actividades Económicas (IAE). Posteriormente, es obligatorio realizar el alta en el Régimen Especial de Trabajadores Autónomos (RETA) a través de la Seguridad Social para regularizar la situación laboral del promotor. Finalmente, dado que se trata de un servicio digital, se debe asegurar el estricto cumplimiento de la normativa vigente en materia de protección de datos (RGPD) y comercio electrónico, garantizando así un entorno seguro para los usuarios finales de la aplicación.

• Viabilidad tecno-económica

El desarrollo de *MineManager FX* se ha ejecutado bajo un modelo de autoempleo (autónomo), lo que ha permitido una ejecución eficiente sin costes operativos iniciales significativos. A nivel de costes de implementación, se ha realizado una inversión en tiempo de desarrollo y una inversión mínima en hardware, utilizando equipos ya disponibles y software de código abierto. El proyecto es altamente escalable y rentable, ya que una vez finalizado el desarrollo, el coste de mantenimiento se limita al alojamiento del servidor de base de datos y al hosting de la página informativa.

La viabilidad a futuro está garantizada mediante una estrategia de monetización no intrusiva. Se contempla la integración de banners publicitarios laterales que no afectan a la jugabilidad, permitiendo financiar el coste de los servicios en la nube (VPS) conforme crezca la base de usuarios. A continuación, se detalla la estimación de costes para un escenario de explotación profesional durante el primer año:

Concepto	Detalle	Coste Estimado (€)
Hardware	Amortización equipo (Base de desarrollo)	450,00 €
Software	Licencias (Entorno de desarrollo y diseño - Open Source)	0,00 €
Infraestructura Cloud	Servidor VPS para BBDD y Ranking (1 año)	120,00 €
Dominio y Web	Dominio .com y hosting básico	30,00 €
TOTAL		600,00 €

Nota: Se han utilizado licencias gratuitas o educativas para todo el software (IntelliJ IDEA, JavaFX, MySQL), lo que reduce el coste de capital (CAPEX) a cero.

• Trabajo futuro

A pesar de que “MineManager Pro” cumple con la totalidad de los requisitos técnicos, arquitectónicos y funcionales propuestos en sus objetivos, la ingeniería de software es un proceso iterativo y evolutivo. De cara a futuras versiones del producto, se plantean tres líneas estratégicas de mejora orientadas a la optimización de la infraestructura, la experiencia del usuario y la escalabilidad del sistema:

14.1 Migración a una Arquitectura Distribuida (Cliente-Servidor)

La mejora de infraestructura más crítica consiste en desacoplar por completo la persistencia del entorno local de la máquina cliente para asegurar un entorno centralizado:

API REST Backend: Se propone el desarrollo de un backend centralizado utilizando frameworks industriales como Spring Boot o Node.js, el cual gobernaría de manera segura el acceso directo a la base de datos relacional MySQL alojada en un servidor en la nube.

Seguridad y Comunicación: El juego pasaría a actuar como un cliente ligero que se comunicaría mediante peticiones HTTP asíncronas intercambiando estructuras JSON. La autenticación local se sustituiría por un control de sesiones basado en Tokens JWT (JSON Web Tokens) cifrados, lo que garantizaría un ecosistema de datos común y blindaría el software frente a la manipulación local de los registros del histórico por parte de usuarios maliciosos.

14.2 Migración Tecnológica del Frontend hacia Entornos Web y Móviles

Para responder a las demandas del mercado actual, se proyecta expandir la disponibilidad multiplataforma de la aplicación fuera del entorno nativo de escritorio:

Entorno Web: Se plantea portar la interfaz de usuario hacia frameworks SPA modernos como Angular o React, aprovechando las facilidades del diseño adaptativo (Responsive Design) mediante Tailwind CSS para su ejecución en navegadores.

Entorno Móvil: Adaptar el motor algorítmico y visual mediante Kotlin Multiplatform o Flutter para generar compilados nativos en Android e iOS, transformando los eventos de clic de ratón actuales (MouseButton) en gestos táctiles optimizados con respuesta háptica.

14.3 Evolución de Mecánicas, Personalización Estética y Accesibilidad

Con el objetivo de maximizar la retención de usuarios y el pulido del producto, se contemplan expansiones directas sobre las reglas de negocio y los componentes de la UI:

Ampliación de Contenido y Modos: Diseñar e inyectar nuevos tipos de logros automatizados en el backend y habilitar modalidades de juego disruptivas que rompan la ortoganilidad clásica del mapa, permitiendo generar tableros basados en geometrías no rectangulares (como matrices hexagonales o triangulares).

Gamificación y Personalización del Perfil: Evolucionar la entidad Usuario para permitir la carga y almacenamiento de avatares dinámicos. Este recurso se indexaría directamente en las consultas de la API Criterias para renderizar la imagen del jugador de forma gráfica junto a su récord en el ranking global.

Ajustes Avanzados de Entorno y Temas Visuales: Enriquecer el panel de configuraciones permitiendo la parametrización de hilos de música de fondo diferenciados para los estados de tensión y el intercambio en caliente de hojas de estilos (CSS). Esto permitiría al usuario elegir entre diferentes temas visuales para el tablero (retro, moderno o futurista).

Inclusión y Accesibilidad Universal: Programar un módulo de conversión de color para activar un Modo daltónico sobre los nodos de la interfaz. Este mecanismo alteraría dinámicamente las paletas de color numéricas y de alerta, garantizando que el software cumpla de manera estricta con las pautas internacionales de accesibilidad web y de aplicaciones.

• Conclusiones

Tras la finalización del periodo de desarrollo de *MineManager FX*, se puede afirmar que los objetivos planteados en el anteproyecto inicial se han cumplido satisfactoriamente. Se ha logrado desarrollar una plataforma que no solo integra la lógica clásica del juego Buscaminas, sino que la eleva mediante un sistema de persistencia en base de datos y un módulo de gamificación que incentiva la superación de marcas personales. La arquitectura seleccionada, basada en JavaFX y una base de datos relacional MySQL, ha demostrado ser robusta y eficiente para un entorno de escritorio, permitiendo una gestión de estados de partida y registros de usuarios coherente con los requisitos técnicos definidos.

Es importante destacar que el proceso de desarrollo ha permitido aplicar de forma práctica los conocimientos adquiridos en el ciclo, especialmente en materias como Acceso a Datos y Desarrollo de Interfaces. Si bien la totalidad de las funcionalidades planificadas se han implementado con éxito, ciertos aspectos relacionados con la optimización de los algoritmos de generación recursiva del tablero presentaron desafíos imprevistos durante la fase de pruebas, lo cual obligó a realizar ajustes que, si bien incrementaron el tiempo de desarrollo, resultaron en una estabilidad superior de la aplicación final.

Como reflexión final, este proyecto ha servido para consolidar competencias en el manejo de persistencia de datos y en la arquitectura MVC, facilitando una estructura que, aunque funcional, tiene un margen de mejora claro. Se han cumplido los plazos establecidos y la aplicación resultante es un producto software terminado, con un diseño intuitivo y un sistema de administración que permite gestionar el flujo de datos de manera autónoma, cumpliendo así con las expectativas de funcionalidad trazadas al inicio del ciclo.

• Biblioteca de recursos web y referencias

La mayor parte de los recursos gráficos (imágenes, iconos) y sonoros empleados en *MineManager FX* son de creación propia o provienen de repositorios bajo licencia Creative Commons Zero (CC0), permitiendo su uso comercial y modificación sin necesidad de atribución obligatoria. A continuación, se detallan las fuentes de información técnica y recursos externos consultados para la elaboración de este proyecto:

Oracle Corporation (2026): JavaFX Documentation, Oracle Java Documentation, disponible en: <https://openjfx.io/javadoc/>

Oracle Corporation (2026): MySQL 8.0 Reference Manual, Oracle MySQL Documentation, disponible en: <https://dev.mysql.com/doc/refman/8.0/en/>

Visual Paradigm (2026): UML Class Diagram Tutorial, Visual Paradigm Guide, disponible en: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

Nota: Todas las tablas y diagramas incluidos en la presente memoria han sido de elaboración propia, basándose en el análisis técnico realizado durante el desarrollo del proyecto en el repositorio antonfaraldo/proyecto-fin-de-ciclo_59. Los recursos multimedia de terceros cuentan con licencias CC0 que permiten su libre uso y modificación sin atribución obligatoria.